

Technical Design Documentation

[Jump to bottom](#)

Emil Badura edited this page 18 hours ago · [3 revisions](#)

Technical/Design Documentation

This document breaks down the technical implementation of OptiBot's real-time object detection pipeline. It explains how we stream video via WebRTC, process frames with YOLO and MiDaS, and wire everything together.

1. Architecture

The system follows a three-tier architecture with microservices connected via WebRTC. The system consists of multiple microservices communicating with each other via WebRTC or WebSocket. We differentiate between 4 microservice types:

- The **Streamer service** provides a video source, this can be a live webcam recording or just a video file.
- The **Analyzer service** receives the stream, performs YOLO object detection/segmentation and depth estimation, and forwards only the resulting metadata about a frame.
- The **Orchestrator service** exists usually just once per system and serves as a middle man between streamer/analyzer and the frontend.
- The **Frontend** displays the selected video stream + corresponding metadata in the browser.

For detailed architecture diagrams and explanations, see the [Software Architecture Document](#).

2. AI models

At the core of our processing pipeline are two AI model types.

First we have our models for object detection or segmentation for more precise object boundaries. For that we use models from the YOLO family, default is the YOLO11n model since it performs well on many [benchmarks](#) while still being quite fast. The model outputs bounding boxes with class IDs and confidence scores for 80 COCO object classes, while the segmentation version (-seg) additionally creates a binary mask for each prediction. We cache inference results to maintain smooth frame rates. See

[src/backend/common/core/detector.py](#).

For distance estimation, we have three model families one can choose from:

- models from the [MiDaS family](#) (default is MiDaS small), which provide only relative coordinates and may be therefore inaccurate
- models from the [Depth Anything V2 family](#) (default is Depth-Anything-V2-small), which provides monocular metric depth while being still quite fast
- models from Apples [Depth Pro family](#), which also provide monocular metric depth, but require better hardware

For each image we create a depth map using these models and sample certain regions where the objects are supposed to be, based on the object detection bounding boxes or the more accurate segmentation masks, if available. We also have an experimental scale factor `SCALE_FACTOR` or `DEPTH_ANYTHING_SCALE_FACTOR`, which can be configured, depending on the camera. See [src/backend/common/utils/depth.py](#) for the concrete implementation.

Both models are initialized once at startup.

3. Orchestrator service: Managing und connecting services

This is the first service one should start and one needs only one instance of it. The analyzer and streamer services know its address and port and register themselves to it (and unregister in case of a shutdown), so that the orchestrator service is able to keep an active list of both of them. A frontend service can connect with the orchestrator via WebSocket to get the list of streamers so that the user can switch between them effortlessly. During the startup, the frontend sends an `/assign` request to the orchestrator, which then chooses the first available analyzer service from its list and returns both the streamer service and analyzer service URLs, so that the frontend can connect to both the analyzer and streamer service to get the correct data streams. The frontend uses the `/assign` operation again during the switch, but before that it does an `/unassign` operation, so that the analyzer can be "freed" from the streamer service and be reused later for the same or for a different frontend instance.

4. Streamer service: Capturing and streaming

The streamer service provides the source for all kinds of video feed and there are two variants:

- `streamer-webcam`, which captures the first available webcam on the system
- `streamer-file`, which opens a video file (usually .mp4) using the `VIDEO_FILE_PATH` in the configuration

At the start, the streamer service registers itself to the orchestrator service. Both variants provide the video source via WebRTC to other services like the analyzer or the frontend service. For that we use an async background loop to continuously read frames. The camera capture is managed with reference counting to support multiple consumers. When a client connects, the service creates a WebRTC connection and streams frames. Each frame is flipped horizontally for mirror effect, converted from BGR to RGB, and encoded to H.264 for efficient streaming.

See [src/backend/streamer/](#) for implementation.

5. Analyzer service: Receiving and processing

An analyzer service is another consumer of one streamer service at a time and forwards its results to a frontend instance. This allows us to keep the advantage of WebRTC delivering fast raw video streams to multiple clients simultaneously while sending the resulting metadata also quite fast via WebSocket to the frontend.

At startup, the analyzer also registers to the orchestrator so that it can be assigned later to a certain streamer component and frontend component by the orchestrator, if one frontend performs the mentioned `/assign` operation. The analyzer establishes a WebRTC connection to the webcam service to receive raw frames. It then wraps this incoming stream with AI processing (YOLO detection and MiDaS depth estimation), renders the overlays, and streams the metadata containing bounding boxes, confidence scores, class labels and depths via WebSocket. It remains idle while unassigned and waits for the configuration from the orchestrator, although the models are still loaded so that it can start immediately without needing a warmup.

See [src/backend/analyzer/](#) for implementation.

6. Drawing detection overlays

Once the analyzer runs object detection and depth estimation, it renders the results directly onto video frames before streaming to the browser. The implementation uses OpenCV's drawing functions. For each detection, `draw_detections()` draws a green rectangle using `cv2.rectangle()` and adds a label with format `"{class_id}:{confidence:.2f}{distance:.1f}m"` using `cv2.putText()`.

Source code <src/backend/common/utils/depth.py> .

7. Frame transformation pipeline

As frames flow through the services, they undergo several transformations to match the requirements of different libraries and protocols.

The **streamer service** captures raw frames from the camera or a video file, flips them for a mirror effect if necessary, and converts them to the color format expected by WebRTC. The frames are then encoded and streamed to the analyzer service.

The **analyzer service** receives the stream, converts frames to the format expected by the AI models, runs object detection and depth estimation, draws overlays on the frames, and streams the annotated video to the browser.

The **frontend** receives the encoded stream, decodes it, and renders it in a standard video element.

The main transformations are color space conversions to match different library expectations, format conversions between video and array representations, and video encoding for efficient network transmission. See <src/backend/streamer/tracks.py> and <src/backend/analyzer/manager.py> .

8. Frontend: Displaying the stream

The pipeline ends with the frontend displaying the annotated video stream. The browser establishes a WebRTC connection to receive the video stream from the analyzer. It negotiates the connection by exchanging session details with the server, then receives the video track and displays it in a standard HTML `<video>` element. It gets the needed analyzer URL from the orchestrator service by performing an `/assign` request for a given streamer URL, while the streamer URL list is constantly being updated through the open WebSocket connection with the orchestrator.

See <src/frontend/src/hooks/useWebRTCPlayer.ts> , <src/frontend/src/hooks/useOrchestrator.ts> and [VideoPlayer.tsx](#) .

▼ Pages 5

Find a page or section...

▸ [Home](#)

▸ [Build Documentation](#)

▸ [Fine Tuning](#)

▼ **Technical Design Documentation**

Technical/Design Documentation

1. Architecture
2. AI models
3. Orchestrator service: Managing und connecting services
4. Streamer service: Capturing and streaming
5. Analyzer service: Receiving and processing
6. Drawing detection overlays
7. Frame transformation pipeline
8. Frontend: Displaying the stream

► **User Documentation**

Clone this wiki locally

<https://github.com/amosproj/amos2025ws04-robot-visual-perception.wiki.git>

